ChatGPT

RAG 분야별 대표 논문 지도: 전체 구조·핵심 쟁점·연구 흐름

Executive summary

RAG를 논문 중심으로 공부할 때 가장 중요한 것은 **"RAG 논문"이라는 하나의 분야를 읽는 것이 아니라, Information Retrieval · Representation Learning · ANN · Document AI · Ranking · LLM Generation · Evaluation · Systems · Security가 합쳐진 스택을 읽는 것**이다. Microsoft의 현재 RAG 설계 가이드도 데이터 준비 → chunking → enrichment → embedding → index/search → retrieval → LLM evaluation → agentic RAG를 각각 독립적인 설계·평가 단계로 취급한다. [1]

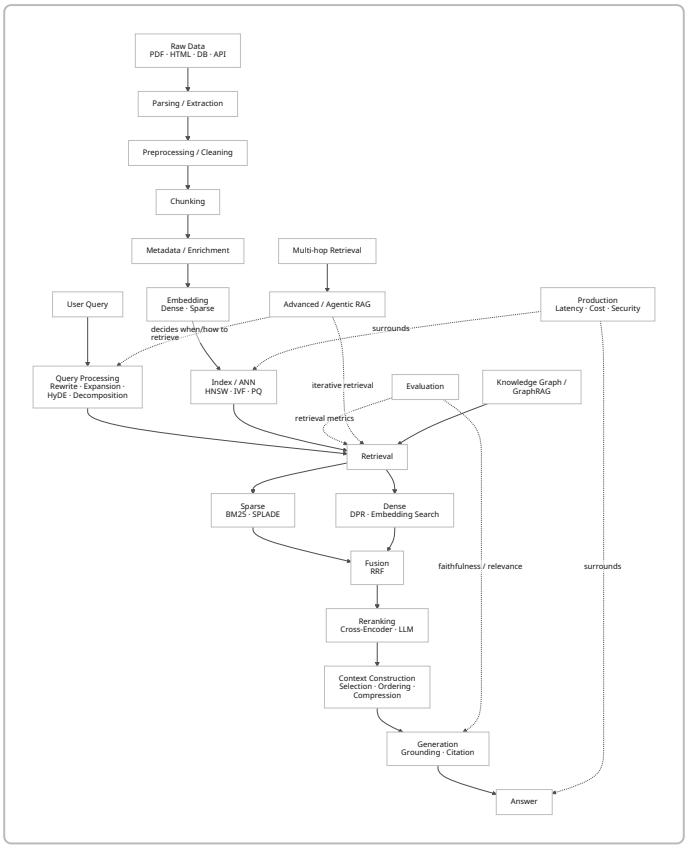
이 보고서에서 가장 우선순위를 높게 둔 축은 **Embedding → IR/BM25 → ANN → Retrieval → Reranking → Chunking → Evaluation**이다. 이유는 이 축을 이해해야 "왜 벡터 검색만으로 부족한가", "HNSW는 embedding 모델과 어떤 관계인가", "retrieval과 reranking은 왜 분리하는가", "chunk 크기가 Recall@K에 어떤 영향을 주는가" 같은 RAG 설계의 핵심 질문을 설명할 수 있기 때문이다. DPR · SPLADE · HNSW · BM25/BEIR · BERT reranker가 이 뼈대를 만든다. [2]

그다음 연구 흐름은 **"한 번 검색해서 top-k를 LLM에 넣는다"는 Naive RAG의 가정을 하나씩 깨는 방향**으로 발전했다. HyDE/Query2doc는 검색 전 query를 바꾸고, RRF는 여러 retriever를 융합하며, reranker는 후보 순위를 다시 계산하고, RECOMP는 context를 압축하며, Self-RAG/Adaptive-RAG/CRAG는 아예 "검색할지, 다시 검색할지, 검색 결과를 믿을지"를 동적으로 결정한다. [3]

또 하나의 중요한 결론은 **RAG의 각 분야가 같은 성숙도를 갖지 않는다는 것**이다. Embedding · IR · ANN · reranking은 수십 년의 정보검색 연구 위에 서 있지만, RAG-specific metadata enrichment, agentic RAG, production security 등은 비교적 최근의 연구 영역이다. 따라서 아래의 "대표 논문 2편"은 모든 분야에서 동일한 의미의 "고전 두 편"이 아니라, **① 해당 기술의 기반을 만든 논문 + ② 현재 RAG 설계에 연결되는 논문**이라는 원칙으로 골랐다. Microsoft 역시 chunking, enrichment, embedding, vector/full-text/hybrid retrieval, evaluation을 별도 실험 대상으로 두도록 권고한다. [4]

전체 구조와 선정 기준

전체 RAG 연구지형은 다음처럼 보는 것이 가장 깔끔하다.



이 그림에서 특히 구분해야 하는 것은 **Embedding과 ANN**, 그리고 **Retrieval과 Reranking**이다. Embedding은 "의미를 어떻게 벡터 공간에 표현할 것인가"의 문제이고, ANN은 이미 만들어진 수백만·수십억 벡터 중 가까운 것을 얼마나 빠르게 찾느냐의 문제다. Retrieval은 대규모 corpus에서 후보군을 싸게 넓게 찾는 단계이고, reranking은 그 후보에 더 비싼 모델을 써서 정밀하게 순서를 다시 매기는 단계다. DPR, HNSW, BERT reranker가 각각 이 세 층을 대표한다. 5

우선순위 표기는 다음과 같이 보면 된다. **★★★★★는 가장 먼저 깊게 읽을 분야**, ★★★★☆는 그다음, ★★★☆☆는 전체 구조를 이해한 뒤 들어갈 분야다. Preprocessing과 Metadata는 RAG-native 고전 논문이 상대적으로 적으므로 인접 분야의 데이터 품질 연구와 최신 RAG-specific 연구를 섞어 선정했다. Production 역시 하나의 알고리즘 분야라기보다 시스템·비용·보안의 결합 영역이다. 4

최우선 분야 핵심 논문

분야	대표 논문	핵심 내용	왜 중요한가·핵심 쟁점	후속 연구·응용 방향
Embedding ★★★★★	**Dense Passage Retrieval for Open-Domain Question Answering** — Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, Wen-tau Yih, **2020**. ACL 원문	DPR은 query encoder와 passage encoder를 분리한 **dual/bi-encoder dense retriever**로, 당시 사실상 표준이던 TF-IDF/BM25 계열 대신 학습된 dense representation만으로 open-domain QA retrieval을 수행할 수 있음을 보였다. Query와 passage를 독립적으로 embedding하므로 문서 벡터를 미리 index할 수 있다. 6	오늘날 Vector RAG의 직접적인 조상에 가깝다. 핵심 쟁점은 **semantic matching 능력 vs lexical exact-match 손실**, 그리고 학습 분포가 달라질 때 dense retrieval이 얼마나 일반화하는가이다. 7	hard-negative mining, contrastive learning, instruction-tuned embedding, multilingual/domain embedding, late interaction, dense+sparse hybrid로 이어진다. BEIR 같은 zero-shot benchmark가 이 일반화 문제를 본격적으로 드러냈다. 8
Embedding ★★★★★	**SPLADE v2: Sparse Lexical and Expansion Model for Information Retrieval** — Thibault Formal, Carlos Lassance, Benjamin Piwowarski, Stéphane Clinchant, **2021**. arXiv	SPLADE는 Transformer를 사용하지만 결과 표현은 dense vector가 아니라 **고차원 sparse lexical representation**이다. 학습된 term expansion을 통해 원문에 없는 관련 단어에 가중치를 주면서도 inverted index 기반 검색의 장점을 유지한다. 9	“embedding = dense vector”라는 생각을 깨준다. sparse representation은 exact token/entity match와 검색엔진 인프라에 강하고, dense embedding은 의미적 paraphrase에 강하기 때문에 현대 RAG의 hybrid retrieval을 이해하는 핵심 연결점이다. 10	SPLADE++, sparse+dense fusion, unified multi-function embeddings, learned sparse retrieval + LLM query expansion이 후속 축이다.

분야	대표 논문	핵심 내용	왜 중요한가·핵심 쟁점	후속 연구·응용 방향
ANN / Indexing ★★★★★	**Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs** — Yu. A. Malkov, D. A. Yashunin, **2020 journal / 2016 preprint**. arXiv	HNSW는 proximity graph를 계층적으로 구성하고 상위 sparse layer에서 빠르게 이동한 뒤 하위 layer에서 세밀하게 탐색하는 ANN 방법이다. 높은 recall과 빠른 query 속도의 조합 때문에 현대 vector search의 핵심 index 구조 중 하나가 됐다. 11	이 논문을 읽으면 `M`, `efConstruction`, `efSearch` 같은 Vector DB 옵션이 단순한 제품 설정이 아니라 **메모리·build time·latency·recall trade-off**임을 이해하게 된다. ANN recall이 낮으면 embedding 모델이 아무리 좋아도 관련 chunk가 후보군에 들어오지 않는다. 12	disk-based ANN, filtered ANN, distributed HNSW, vector compression, dynamic updates, hybrid index 설계로 확장된다.
ANN / Indexing ★★★★★	**Product Quantization for Nearest Neighbor Search** — Hervé Jégou, Matthijs Douze, Cordelia Schmid, **2011**. DOI	PQ는 고차원 벡터를 여러 subspace로 나누고 각각을 quantize하여 짧은 code로 압축한 뒤 approximate distance를 효율적으로 계산한다. 논문은 inverted file과의 결합도 다뤄 이후 IVF-PQ 계열과 대규모 vector search의 중요한 기반이 됐다. 13	HNSW가 주로 **graph navigation** 문제라면 PQ는 **memory/computation compression** 문제를 다룬다. 수억~수십억 벡터가 되면 "정확도 높은 embedding"만큼 벡터당 메모리와 cache locality가 중요해진다. 13	IVF-PQ, OPQ, GPU-accelerated search, Faiss 계열로 이어진다. GPU 대규모 similarity search 연구는 이 계보를 실용적 billion-scale retrieval로 확장했다. 14
Retrieval / IR ★★★★★	**The Probabilistic Relevance Framework: BM25 and Beyond** — Stephen Robertson, Hugo Zaragoza, **2009**. DOI	BM25 계열을 probabilistic relevance 관점에서 체계적으로 설명한 고전이다. Term frequency의 효과를 포화시키고 document length를 보정하는 방식이 왜 단순 TF-IDF보다 강력한 lexical retrieval을 만드는지 이해하는 출발점이다. 15	RAG를 제대로 배우려면 BM25를 건너뛰면 안 된다. 제품 코드, 사람 이름, 버전, 법조문 번호처럼 **정확한 토큰이 의미 자체보다 중요한 query**에서는 dense retrieval만으로 부족할 수 있다는 문제의 출발점이 된다.	BM25F, learned sparse retrieval, SPLADE, dense+sparse hybrid search가 자연스러운 후속이다. Microsoft의 RAG 설계 가이드도 vector, full-text, hybrid를 별도 선택지로 명시한다. 4

분야	대표 논문	핵심 내용	왜 중요한가·핵심 쟁점	후속 연구·응용 방향
Retrieval / IR ★★★★★	**BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models** — Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, Iryna Gurevych, **2021**. arXiv	BEIR은 서로 다른 도메인·task의 IR datasets를 묶어 retriever의 **zero-shot generalization**을 비교하는 benchmark를 제공했다. 특정 QA dataset에서 잘 학습된 dense retriever가 모든 검색 문제에서 BM25를 일관되게 압도하는 것은 아니라는 사실을 연구 커뮤니티가 더 체계적으로 비교할 수 있게 했다. 16	RAG에서 "어떤 embedding model이 제일 좋아요?"라는 질문에 universal answer가 없다는 것을 이해하게 한다. **in-domain 성능과 out-of-domain robustness**, sparse와 dense의 상보성이 핵심 쟁점이다.	MTEB/다국어 retrieval benchmarks, domain adaptation, hybrid retrieval, reranking benchmark로 이어진다.
Reranking ★★★★★	**Passage Re-ranking with BERT** — Rodrigo Nogueira, Kyunghyun Cho, **2019**. arXiv	Query와 passage를 한 Transformer에 함께 넣어 relevance를 직접 예측하는 **cross-encoder reranking**의 대표 초기 연구다. 문서를 독립적으로 미리 encoding하는 bi-encoder보다 계산량은 훨씬 크지만 query-document token interaction을 직접 모델링할 수 있다. 17	여기서 RAG의 전형적인 `retrieve 100 → rerank 10 → LLM 5` 구조가 나온다. **retrieval recall과 reranking precision을 서로 다른 모델로 최적화**한다는 발상이 핵심이다.	MonoT5, cross-encoder distillation, ColBERT/late interaction, multilingual reranker, small reranker 모델로 발전한다.

분야	대표 논문	핵심 내용	왜 중요한가·핵심 쟁점	후속 연구·응용 방향
Reranking ★★★★★	**Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents** — Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, Zhaochun Ren, **2023**. ACL	생성형 LLM을 직접 relevance ranking agent로 사용할 수 있는지를 체계적으로 조사했다. 특히 pointwise/listwise 계열 prompting을 통해 LLM의 language understanding을 ranking에 사용하는 가능성과 비용·ranking objective mismatch 문제를 보여준다. 18	전통적인 작은 cross-encoder와 범용 LLM reranker의 경계를 보여준다. 쟁점은 **품질 향상과 막대한 latency/token cost**, position bias, prompt sensitivity다.	RankGPT, pairwise ranking prompting, distillation을 통해 "LLM의 ranking 지식을 작은 reranker로 이전"하는 방향이 중요하다.
Chunking ★★★★★	**Dense X Retrieval: What Retrieval Granularity Should We Use?** — Tong Chen, Hongwei Wang, Sihao Chen, Wenhao Yu, Kaixin Ma, Xinran Zhao, Hongming Zhang, Dong Yu, **2024**. ACL	Document, passage, sentence보다 더 원자적인 **proposition**을 retrieval unit으로 다루고 retrieval granularity가 downstream QA 성능에 미치는 영향을 분석한다. self-contained proposition 단위가 특히 세부·long-tail 정보를 찾는 데 유리할 수 있음을 보여준다. 19	"chunk size는 512가 좋나요?"라는 질문을 **granularity 문제**로 격상시킨 논문이다. 작은 chunk는 retrieval precision을 높일 수 있지만 context와 관계 정보가 손실된다는 근본 trade-off가 있다.	proposition chunking, parent-child retrieval, small-to-big retrieval, hierarchical retrieval이 후속 방향이다.
Chunking ★★★★★	**Late Chunking: Contextual Chunk Embeddings Using Long-Context Embedding Models** — Michael Günther, Isabelle Mohr, Daniel James Williams, Bo Wang, Han Xiao, **2024**. arXiv	일반 RAG는 먼저 문서를 자른 후 각 chunk를 독립적으로 embedding하지만, Late Chunking은 긴 문서를 먼저 Transformer에 통과시킨 뒤 token representations를 chunk 단위로 pooling한다. 따라서 chunk embedding 자체가 주변 문맥을 어느 정도 보존한다. 20	**"chunk 경계 때문에 의미가 사라진다"**는 고전적 문제를 embedding 단계에서 해결하려는 시도다. chunking과 embedding이 사실 완전히 독립적인 단계가 아니라는 점을 보여준다.	context-aware embedding, long-context embedding, document-level contextual retrieval, hierarchical embedding으로 이어진다.

분야	대표 논문	핵심 내용	왜 중요한가·핵심 쟁점	후속 연구·응용 방향
Evaluation ★★★★★	**RAGAS: Automated Evaluation of Retrieval Augmented Generation** — Shahul Es, Jithin James, Luis Espinosa-Anke, Steven Schockaert, **2023**. arXiv	RAG를 reference answer 하나의 정확도로만 평가하지 않고 **faithfulness, answer relevance, context relevance** 등 component-oriented 관점에서 자동 평가하려는 프레임워크다. 사람 annotation이 충분하지 않은 실제 RAG 실험에서 빠르게 비교할 수 있도록 설계됐다. 21	RAG가 실패했을 때 "retriever가 실패했나, generator가 context를 무시했나"를 분해해서 보는 사고방식을 확산시켰다. 단, LLM-as-a-judge류 metric 자체의 bias와 안정성도 별도로 검증해야 한다. 21	judge calibration, claim-level evaluation, multilingual/domain evaluation, human correlation 분석으로 발전한다.
Evaluation ★★★★★	**ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems** — Jon Saad-Falcon, Omar Khattab, Christopher Potts, Matei Zaharia, **2023**. arXiv	ARES는 synthetic training data로 lightweight judges를 학습해 **context relevance, answer faithfulness, answer relevance**를 평가하고, 소량의 human annotation과 prediction-powered inference를 결합한다. Domain shift 상황에서도 평가 안정성을 연구했다. 22	단순히 GPT에게 "1~5점 줘"라고 하는 평가보다 **평가자 자체를 하나의 통계적 시스템으로 설계**하려는 흐름을 보여준다. Evaluation 비용, judge reliability, domain transfer가 핵심 쟁점이다.	RAGChecker처럼 retrieval과 generation error를 더 세분화하는 diagnostic evaluation으로 이어진다. RAGChecker는 세부 metric을 이용해 여러 RAG architecture의 trade-off를 분석한다. 23

이 여섯 분야를 먼저 읽으면 RAG의 검색부가 사실상 다음 식으로 보이기 시작한다.

$$\text{Corpus} \xrightarrow{\text{chunk}} d_i \xrightarrow{\text{embedding}} v_i \xrightarrow{\text{ANN}} C_{100} \xrightarrow{\text{reranker}} C_5 \xrightarrow{\text{LLM}} Answer$$

여기서 **BM25/SPLADE는 embedding+ANN 경로와 병렬로 존재할 수 있고**, RRF 같은 fusion이 두 후보 목록을 합친다. 이 구조를 이해한 뒤부터 vector database 제품 비교가 의미 있어진다. DB 이름부터 외우는 것보다 HNSW/PQ/BM25/dense-sparse fusion을 먼저 이해하는 편이 훨씬 이식성이 높다. Microsoft도 실제 설계 단계에서 vector, full-text, hybrid, multiple searches를 별도 선택지로 다룬다. 4

파이프라인 전·후단 핵심 논문

분야	대표 논문	핵심 내용	왜 중요한가·핵심 쟁점	후속 연구·응용 방향
Parsing / Extraction ★★★★☆	**LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking** — Yupan Huang, Tengchao Lv, Lei Cui, Yutong Lu, Furu Wei, **2022**. arXiv	텍스트만 읽는 것이 아니라 document image, text, spatial layout을 함께 modeling하는 multimodal Document AI 모델이다. Form understanding, receipt understanding, document VQA, layout analysis 등에서 문서의 2D 구조가 중요한 이유를 보여준다. [24]	PDF RAG에서 extraction을 `pdftotext` 정도로 생각하면 table, column, heading, reading order가 파괴될 수 있다. **layout 자체가 semantic information**이라는 점이 핵심이다.	layout-aware parsers, multimodal document models, vision-language PDF parsing, table-aware RAG로 확장된다.
Parsing / Extraction ★★★★☆	**Nougat: Neural Optical Understanding for Academic Documents** — Lukas Blecher, Guillem Cucurull, Thomas Scialom, Robert Stojnic, **2023**. arXiv	과학 문서 page image를 markup으로 변환하여 수식·표·문서 구조를 보존하려는 neural OCR/document understanding 접근이다. 특히 기존 OCR이 어려워하는 scientific PDF의 mathematical notation과 구조적 변환에 초점을 맞춘다. [25]	Parsing 오류는 이후 chunking·embedding·retrieval 전체에 전파된다. 즉 RAG에서 **garbage in → garbage retrieved**가 일어나므로, upstream document fidelity가 retrieval quality의 일부다.	Docling 같은 end-to-end document conversion, multimodal RAG, table extraction, figure grounding으로 이어진다. [26]
Preprocessing / Cleaning ★★★☆☆	**Deduplicating Training Data Makes Language Models Better** — Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, Nicholas Carlini, **2022**. arXiv	직접적인 RAG 논문은 아니지만 corpus cleaning에 중요한 기반 연구다. 대규모 text corpus에 near-duplicates와 반복 문자열이 상당히 존재하며, deduplication이 memorization 및 evaluation contamination을 줄일 수 있음을 실험했다. [27]	RAG corpus에도 같은 문제가 생긴다. 동일 정책 문서가 여러 위치에 복제되면 top-k가 사실상 같은 chunk들로 채워져 **retrieval diversity와 effective context capacity**가 감소한다는 것이 실무적 쟁점이다. 이는 논문 결과를 RAG에 적용한 추론이다. [27]	exact hash → MinHash/LSH → embedding-based semantic dedup, version-aware document management로 발전시켜 적용할 수 있다.

분야	대표 논문	핵심 내용	왜 중요한가·핵심 쟁점	후속 연구·응용 방향
Preprocessing / Cleaning ★★★☆☆	**The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only** — Guilherme Penedo, Quentin Malartic, Daniel Hesslow, Ruxandra Cojocaru, Alessandro Cappelli, Hamza Alobeidli, Baptiste Pannier, Ebtesam Almazrouei, Julien Launay, **2023**. NeurIPS	대규모 web corpus를 filtering과 stringent deduplication으로 정제하는 pipeline의 중요성을 보여준다. 잘 정제된 web-only data로도 높은 품질의 language-model training corpus를 만들 수 있음을 보였다. 28	역시 RAG 전용 논문은 아니지만, **retrieval corpus quality가 모델 품질 못지않게 중요하다**는 데이터 중심 관점을 배우기에 좋다. spam, boilerplate, obsolete version, contradictory copy 제거가 RAG ingestion에도 그대로 연결된다.	freshness scoring, source trust, document versioning, provenance-aware cleaning을 RAG-specific preprocessing에 결합하는 방향이 중요하다.
Metadata / Enrichment ★★★☆☆	**A Systematic Framework for Enterprise Knowledge Retrieval: Leveraging LLM-Generated Metadata to Enhance RAG Systems** — Pranav Pushkar Mishra, Kranti Prakash Yeole, Ramyashree Keshavamurthy, Mokshit Bharat Surana, Fatemeh Sarayloo, **2026**. arXiv	Chunk에 LLM-generated metadata를 추가하고 chunking·embedding 방식과의 상호작용을 실험적으로 분석한다. content-only representation과 metadata-enriched retrieval을 비교하여 metadata가 retrieval representation 자체를 개선할 가능성을 연구한다. 29	이 분야는 아직 DPR/HNSW처럼 확립된 "고전"이 적다. 핵심 쟁점은 **metadata를 filter용으로만 쓸 것인지 embedding에 넣을 것인지**, LLM-generated metadata의 오류가 검색을 오염시키지 않는지다. 29	query-aware metadata, schema-conditioned retrieval, entity/date/authority fields, ACL/RBAC와 metadata filtering을 결합하는 방향이 중요하다.

분야	대표 논문	핵심 내용	왜 중요한가·핵심 쟁점	후속 연구·응용 방향
Metadata / Enrichment ★★★☆☆	**Structure-Aware RAG: Structured Retrieval Augmented Generation from Noisy Data for Conversational Agents** — Kaiqiao Han, LuAn Tang, Renliang Sun, Peng Yuan, Wei Cheng, Haoyu Wang, Wei Wang, Yizhou Sun, Haifeng Chen, **2026**. arXiv	noisy data를 바로 text chunks로 검색하기보다 table을 structured intermediate representation으로 사용하고 **quality-aware table metadata generation**을 도입한다. 구조와 metadata의 품질을 jointly 고려해 noisy context를 줄이는 접근이다. 30	Metadata는 단순한 `author`, `page_number`를 넘어 **retrieval control plane**이 될 수 있다는 흐름을 보여준다. 생성 metadata의 정확성·정규화·schema consistency가 새로운 품질 변수다.	table-RAG, structured RAG, SQL+vector hybrid, metadata gating, schema-aware agents로 연결된다.
Query Processing ★★★★☆	**Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE)** — Luyu Gao, Xueguang Ma, Jimmy Lin, Jamie Callan, **2023**. ACL	Query를 직접 embedding하지 않고 LLM이 먼저 **hypothetical relevant document**를 생성한 후 그 문서를 embedding하여 실제 corpus를 검색한다. 질문 표현과 document 표현 간의 semantic/form mismatch를 줄이는 것이 핵심 아이디어다. 31	Query representation 자체가 retrieval 성능의 bottleneck이라는 점을 보여준다. 다만 hypothetical answer가 hallucinate하면 retrieval 방향이 잘못될 수 있다는 trade-off가 있다.	query rewriting, pseudo-relevance feedback, synthetic query/doc generation, adaptive HyDE로 확장된다.
Query Processing ★★★★☆	**Query2doc: Query Expansion with Large Language Models** — Liang Wang, Nan Yang, Furu Wei, **2023**. arXiv	LLM이 query에 대응되는 pseudo-document를 만들고 이를 query expansion에 사용한다. 저자들은 BM25 및 dense retrieval에 적용하여 단순 query보다 풍부한 retrieval signal을 얻는 방식을 연구했다. 32	HyDE와 함께 보면 "LLM을 답변 생성기에만 쓸 필요가 없다"는 것을 이해한다. **retrieval 전에 LLM을 사용하는 비용 vs recall 향상**이 쟁점이다.	multi-query generation, rewrite-retrieve-read, query routing, decomposition, learned query refinement로 이어진다.

분야	대표 논문	핵심 내용	왜 중요한가·핵심 쟁점	후속 연구·응용 방향
Fusion / RRF ★★★★☆	**Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods** — Gordon V. Cormack, Charles L. A. Clarke, Stefan Büttcher, **2009**. Google Research	서로 score scale이 다른 여러 ranked list를 raw score normalization 없이 **rank만으로 결합**하는 Reciprocal Rank Fusion을 제안했다. 간단한 식임에도 heterogeneous retrievers의 결과를 안정적으로 합치는 방법으로 널리 쓰이게 된 고전이다. 33	Dense cosine score와 BM25 score는 서로 직접 비교하기 어렵다. RRF는 이 **score calibration 문제를 rank-space에서 우회**하기 때문에 hybrid RAG에 특히 자연스럽다.	weighted RRF, learned fusion, query-dependent fusion, metadata-conditioned fusion으로 이어진다.
Fusion / RRF ★★★☆☆	**RAG-Fusion: a New Take on Retrieval-Augmented Generation** — Zackary Rackauckas, **2024**. arXiv	하나의 query에서 여러 변형 query를 생성하고 각각 검색한 뒤 RRF로 결과를 합치는 RAG-specific 응용을 제시한다. 여러 관점의 query가 원 질문의 표현 한계를 보완하지만, 잘못된 query diversification은 off-topic 문서를 끌어올릴 수 있다. 34	원래 2009년 rank-fusion 기법이 LLM 기반 multi-query RAG에서 다시 중요해지는 과정을 보여주는 연결 논문이다. 다만 원 RRF 논문에 비해 이 논문은 **응용 성격의 preprint**라는 점을 감안해 읽는 것이 좋다. 34	learned query weighting, intent-aware fusion, sparse+dense+graph ensemble로 확장된다.
Context Construction ★★★★☆	**Lost in the Middle: How Language Models Use Long Contexts** — Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, Percy Liang, **2024**. TACL	긴 context를 지원하는 LLM도 relevant information의 위치에 따라 성능이 달라질 수 있으며, 정보가 context 중간에 있을 때 성능이 떨어지는 패턴을 보여준다. Multi-document QA와 key-value retrieval을 통해 long-context utilization 자체를 분석했다. 35	`top_k`를 계속 늘리는 것이 항상 좋은 RAG가 아니라는 근거가 된다. **retrieval recall 증가 vs distractor 증가 vs context-position effects**를 동시에 봐야 한다.	context ordering, relevance-aware packing, document dedup, long-context training, context pruning으로 이어진다.

분야	대표 논문	핵심 내용	왜 중요한가·핵심 쟁점	후속 연구·응용 방향
Context Construction ★★★★☆	**RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation** — Fangyuan Xu, Weijia Shi, Eunsol Choi, **2024; preprint 2023**. arXiv	Retrieved documents를 그대로 넣는 대신 extractive 또는 abstractive compressor로 필요한 정보만 축약하고, 도움이 되지 않는 retrieval 결과에는 빈 context를 반환하는 selective augmentation을 제안한다. 논문은 상당한 압축에도 downstream 성능 손실을 제한할 수 있음을 보고한다. 36	RAG의 context 단계가 단순 concatenation이 아니라 **정보 병목 설계**라는 것을 보여준다. Compression이 지나치면 근거가 손실되고, abstractive compression은 새로운 hallucination source가 될 수 있다.	query-aware compression, token-budget optimization, extractive evidence selection, learned context routing으로 발전한다.
Generation / Grounding ★★★★☆	**Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks** — Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, Douwe Kiela, **2020**. arXiv	오늘날 “RAG”라는 명칭을 정립한 핵심 논문으로, seq2seq model의 parametric memory와 Wikipedia dense index의 non-parametric memory를 결합한다. RAG-Sequence와 RAG-Token 두 formulation을 비교하며 retrieval과 generation을 확률적으로 연결했다. 37	RAG가 왜 단순 search+prompt보다 더 넓은 연구개념인지 이해하는 출발점이다. **지식 업데이트 가능성, provenance, retriever-generator coupling**이 핵심 문제로 제시된다. 38	Atlas, RETRO, retrieval-aware fine-tuning, Self-RAG, retrieval-integrated reasoning 등으로 이어진다.

분야	대표 논문	핵심 내용	왜 중요한가·핵심 쟁점	후속 연구·응용 방향
Generation / Citation ★★★★☆	**Enabling Large Language Models to Generate Text with Citations** — Tianyu Gao, Howard Yen, Jiatong Yu, Danqi Chen, **2023**. ACL	장문의 LLM answer에 citation을 생성하고, citation이 실제 주장에 대해 올바른 support를 제공하는지를 평가하는 ALCE 계열 연구다. 단순 answer correctness를 넘어 **attribution/citation correctness와 completeness**를 별도 대상으로 삼는다. 39	"답이 맞다"와 "답의 각 주장이 retrieval evidence에 의해 뒷받침된다"는 다른 문제다. Enterprise/legal/scientific RAG에서는 후자가 특히 중요하다.	claim-level attribution, citation entailment checking, source quality weighting, automatic claim verification으로 발전한다.

여기까지의 연구 흐름을 한 문장으로 요약하면 다음과 같다.

> **좋은 RAG는 relevant text를 찾는 시스템이 아니라, 원본 문서를 제대로 구조화하고 → 질문을 검색 가능하게 바꾸고 → 여러 검색 신호를 결합하고 → 가장 좋은 근거만 정밀하게 선택하고 → LLM이 실제로 활용 가능한 형태로 context를 구성하는 시스템이다.** Microsoft의 현재 설계 가이드도 chunk cleaning/enrichment, embedding, vector/full-text/hybrid retrieval, step-wise evaluation을 각각 독립적으로 최적화해야 할 대상으로 둔다. 4

고급 구조와 운영 핵심 논문

분야	대표 논문	핵심 내용	왜 중요한가·핵심 쟁점	후속 연구·응용 방향
Production / Latency / Cost ★★★☆☆	**RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation** — Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, Xin Jin, **2024**. arXiv	RAG에서 동일·유사 knowledge가 반복적으로 context에 들어갈 때 LLM의 intermediate states를 multi-level cache에 저장하여 prefill 비용을 줄이는 시스템 연구다. Knowledge tree와 GPU/host-memory caching, retrieval/inference overlap 등을 사용해 latency와 throughput을 개선한다. 40	연구용 RAG와 production RAG의 차이를 보여주는 좋은 논문이다. 실제 병목은 vector search 수십 ms보다 **LLM prefill, long retrieved context, memory transfer**에 있을 수도 있다. 41	semantic cache, retrieval cache, KV-prefix sharing, batching, speculative retrieval, cost-aware routing이 후속 방향이다.

분야	대표 논문	핵심 내용	왜 중요한가·핵심 쟁점	후속 연구·응용 방향
Production / Security ★★★☆☆	**PoisonedRAG: Knowledge Poisoning Attacks to Retrieval-Augmented Generation of Large Language Models** — Wei Zou, Runpeng Geng, Binghui Wang, Jinyuan Jia, **2025 USENIX Security; preprint 2024**. arXiv	공격자가 knowledge base에 조작 문서를 삽입해 특정 query에서 악성 passage가 높은 순위로 검색되고 결국 LLM 답변까지 조작되도록 하는 **knowledge poisoning** 공격을 연구한다. 이후 USENIX Security 2025 문헌으로 출판되었다. 42	RAG가 모델 weight를 수정하지 않고도 외부 corpus를 통해 공격받을 수 있음을 보여준다. 문서 access control만으로는 부족하고 **source provenance, ingestion validation, retrieval robustness**가 필요하다.	certified robust RAG, poisoned-document detection, trust scoring, signed provenance, prompt-injection isolation, tenant ACL이 중요한 후속 분야다. 43
Advanced RAG ★★★★☆	**Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection** — Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, Hannaneh Hajishirzi, **2024**. arXiv	모든 질문에서 고정된 top-k를 검색하지 않고 모델이 retrieval 필요성, passage relevance, generation quality 등을 **reflection tokens**으로 판단하도록 학습한다. Retrieval과 generation, critique를 하나의 adaptive inference process로 묶는다. 44	Naive RAG의 가장 큰 가정인 “항상 검색하고 항상 결과를 믿는다”를 깨는 대표 연구다. 다만 이런 self-reflection이 실제로 안정적인지, 추가 inference cost가 정당한지가 쟁점이다.	critic models, learned retrieval policy, reinforcement learning 기반 search/reasoning, agentic RAG로 이어진다.
Advanced RAG ★★★★☆	**Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity** — Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, Jong C. Park, **2024**. arXiv	Query complexity를 분류한 뒤 **no retrieval / single-step retrieval / iterative retrieval** 중 적절한 전략을 선택한다. 쉬운 질문에 비싼 multi-hop retrieval을 쓰거나, 복잡한 질문에 단일 검색만 하는 비효율을 줄이려는 접근이다. 45	RAG architecture 자체를 query별로 선택하는 **routing problem**으로 본다는 점이 중요하다. 정확성뿐 아니라 computation budget과 latency가 의사결정 변수로 들어온다.	CRAG는 retrieved evidence 품질을 평가하여 corrective action을 선택하는 방향을 제안했고, agentic RAG는 retrieval을 필요할 때 호출하는 tool로 더 일반화한다. 46

분야	대표 논문	핵심 내용	왜 중요한가·핵심 쟁점	후속 연구·응용 방향
Graph RAG / KG ★★★★☆	**From Local to Global: A Graph RAG Approach to Query-Focused Summarization** — Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, Jonathan Larson, **2024**. Microsoft Research	문서에서 entity와 relation을 추출해 graph를 만들고 community를 구성한 뒤 community summary를 미리 생성한다. 일반 vector RAG가 어려워하는 "이 corpus 전체의 주요 테마는 무엇인가?" 같은 **global sensemaking query**를 query-focused summarization으로 처리한다. 47	Vector RAG는 query와 특정 passage의 local relevance에는 강하지만 corpus-level structure에는 약할 수 있다. 대신 GraphRAG는 **index 구축 비용, entity/relation extraction 오류, graph maintenance**가 새로운 비용이 된다.	local+global search, dynamic graph construction, temporal KG, graph-agent routing으로 이어진다.
Graph RAG / KG ★★★★☆	**HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models** — Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, Yu Su, **2024**. arXiv	LLM, knowledge graph, Personalized PageRank를 결합해 연관된 지식을 graph를 따라 연결하는 retrieval memory를 제안한다. 특히 multi-hop QA에서 iterative retrieval을 여러 번 호출하지 않고도 associative retrieval을 수행하는 방향을 탐구한다. 48	Graph가 단순 visualization이 아니라 **retrieval index 자체**가 될 수 있음을 보여준다. Dense similarity와 symbolic/graph connectivity 중 어떤 정보를 언제 사용할지가 핵심 쟁점이다.	HippoRAG 2는 non-parametric continual memory 쪽으로 확장됐으며, graph + vector + iterative reasoning 결합이 중요한 후속 축이다. 49

분야	대표 논문	핵심 내용	왜 중요한가·핵심 쟁점	후속 연구·응용 방향
Multi-hop Retrieval ★★★★☆	**Answering Complex Open-Domain Questions with Multi-Hop Dense Retrieval** — Wenhan Xiong, Xiang Lorraine Li, Srini Iyer, Jingfei Du, Patrick Lewis, William Yang Wang, Yashar Mehdad, Wen-tau Yih, Sebastian Riedel, Douwe Kiela, Barlas Oğuz, **2021**. arXiv	첫 검색 결과를 다음 검색 단계의 정보로 사용하여 여러 hop에 걸쳐 evidence를 찾는 dense retriever를 제안한다. Hyperlink나 human entity annotation 같은 corpus-specific 구조 없이 unstructured corpus에 적용하는 것을 목표로 했다. 50	Query 하나의 embedding으로 모든 필요한 evidence를 한 번에 찾을 수 있다는 single-hop 가정을 깨준다. 문제는 hop이 늘수록 **error propagation과 retrieval cost**가 커진다는 것이다.	iterative retrieval, query decomposition, graph traversal, search policy learning으로 발전한다.
Multi-hop Retrieval ★★★★☆	**Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions (IRCoT)** — Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, Ashish Sabharwal, **2023**. ACL	고정된 query로 여러 문서를 가져오는 대신, reasoning step을 생성하고 그 중간 reasoning을 다음 retrieval query로 사용한다. 즉 **retrieve → reason → retrieve → reason**을 교차시켜 검색과 추론이 서로를 안내하도록 한다. 51	현대 Agentic RAG를 이해하는 핵심 전조다. 검색을 하나의 preprocessing step이 아니라 **reasoning loop 내부 action**으로 바꿨다는 점이 중요하다.	search agents, tool-use RL, adaptive stopping, parallel multi-hop search, graph-guided reasoning으로 연결된다.

Advanced RAG 흐름은 특히 아래처럼 이해하면 쉽다.

```
Naive RAG
Query → Retrieve once → Generate
                │
                ├── 문제: 검색이 필요 없는 질문에도 검색
                ├── 문제: 검색 결과가 틀려도 사용
                └── 문제: 복잡한 질문도 한 번만 검색
                      ↓

Active / Adaptive Retrieval
"지금 검색이 필요한가?"
                      ↓

Self-RAG
"검색할까? 이 문서는 유용한가? 내 답은 근거 있는가?"
```

```
                        ↓

CRAG
"검색 결과가 나쁘면 수정/대체 검색하자."
                        ↓

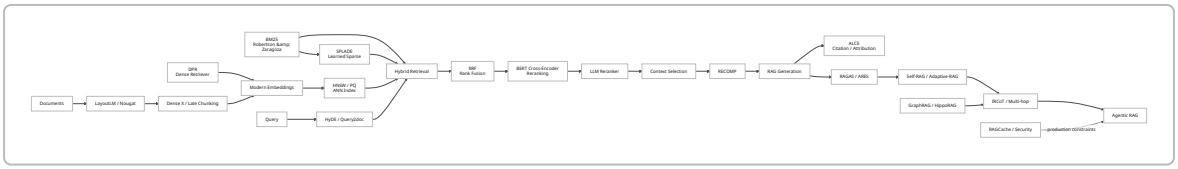
Multi-hop / IRCoT
"중간 추론 결과를 가지고 또 검색하자."
                        ↓

Agentic RAG
"어떤 tool / DB / search를 몇 번 호출할지 동적으로 결정하자."
```

Self-RAG는 on-demand retrieval과 self-critique를 모델 내부 정책으로 가져왔고, CRAG는 별도 retrieval evaluator를 이용해 부정확하거나 애매한 검색 결과에 corrective action을 적용한다. Adaptive-RAG는 query complexity에 따라 retrieval complexity를 조절한다. Microsoft가 현재 설명하는 agentic RAG는 이를 더 일반화해, agent가 retrieval을 "필요할 때 호출하는 도구"로 취급하고 source selection·query decomposition·multi-step retrieval을 runtime에 결정하는 구조다. [52]

분야 간 연관성과 연구 흐름

논문들을 연도별로만 읽으면 서로 별개로 느껴지지만, **각 연구는 앞 단계의 bottleneck을 다음 단계가 보완하는 구조**로 연결된다.



연구의 흐름은 대략 **다섯 번의 관점 변화**로 정리할 수 있다.

첫 번째는 lexical retrieval에서 learned retrieval로의 변화다. BM25는 lexical matching의 강력한 기준점을 제공했고, DPR은 semantic dense retrieval을 실용적인 open-domain QA에 정착시켰다. SPLADE는 두 세계 사이에서 learned semantics를 sparse lexical representation으로 표현한다. BEIR는 특정 benchmark에서의 승리보다 domain이 달라졌을 때 무엇이 살아남는지를 묻게 만들었다. [53]

두 번째는 "좋은 embedding"에서 "좋은 retrieval system"으로의 변화다. 실제 검색은 representation만으로 끝나지 않는다. HNSW/PQ가 latency-memory-recall을 결정하고, BM25와 dense 결과를 RRF로 합치고, cross-encoder가 final relevance를 다시 판단한다. 이 때문에 실무 RAG는 흔히 하나의 similarity search가 아니라 **candidate generation → fusion → reranking cascade**가 된다. [54]

세 번째는 "검색한 문서를 LLM에 넣으면 된다"에서 context engineering으로의 변화다. Lost in the Middle은 긴 context를 제공한다고 해서 모든 evidence가 동일하게 사용되지 않음을 보여주며, RECOMP는 아예 retrieval과 generation 사이에 learned compression layer를 둔다. Chunking 연구 역시 단순 문자열 분할에서 proposition-level granularity와 contextual chunk embedding으로 이동하고 있다. [55]

네 번째는 fixed pipeline에서 adaptive pipeline으로의 변화다. HyDE와 Query2doc는 retrieval 이전에 LLM을 사용하고, Self-RAG·CRAG·Adaptive-RAG는 retrieval 자체를 conditionally 실행·검사·교정한다. IRCoT는

reasoning 중간에 다시 retrieval을 삽입한다. 즉 최신 흐름에서 RAG는 `Retriever → LLM`이라는 직선형 pipeline이라기보다 **search policy를 가진 iterative system**에 가까워지고 있다. 56

다섯 번째는 answer accuracy에서 provenance와 system reliability로의 변화다. ALCE는 citation/attribution을, RAGAS와 ARES는 faithfulness/context relevance 등을, PoisonedRAG는 retrieval corpus 자체가 공격 표면이 되는 문제를, RAGCache는 inference cost를 다룬다. RAG가 production system이 되면서 "정답을 맞혔는가?"만큼 **왜 그 답을 했는지, 어떤 문서에서 왔는지, 얼마나 싸고 빠르며 안전한지**가 연구대상이 된다. 57

특히 공부하면서 다음 쟁점들은 계속 추적하면 좋다.

근본 쟁점	서로 경쟁하는 선택
검색 단위	document ↔ passage ↔ sentence ↔ proposition
representation	sparse ↔ dense ↔ multi-vector ↔ graph
검색 정확도/속도	exact search ↔ ANN
ANN 구조	graph/HNSW ↔ inverted file/IVF ↔ compressed/PQ
retrieval 방식	BM25 ↔ learned sparse ↔ dense ↔ hybrid
후보 판단	retriever score ↔ cross-encoder ↔ LLM reranker
query 처리	original ↔ rewrite ↔ expansion ↔ HyDE ↔ decomposition
context 구성	more context ↔ selective evidence ↔ compression
generation	free generation ↔ grounded generation ↔ citation-constrained generation
pipeline	fixed RAG ↔ adaptive RAG ↔ iterative/agentic RAG
knowledge structure	flat chunks ↔ hierarchy ↔ knowledge graph
평가	answer-only ↔ retrieval+generation 분리 ↔ claim/attribution-level
production	quality ↔ latency ↔ token cost ↔ memory ↔ security

이 표의 핵심은 **"더 최신 기법이 이전 기법을 제거한다"가 아니라 trade-off 공간이 계속 넓어진다**는 것이다. 예를 들어 dense retrieval이 등장했다고 BM25가 사라진 것이 아니고, long-context model이 등장했다고 chunking/retrieval이 사라진 것도 아니다. 오히려 Microsoft의 현재 RAG 설계 가이드는 여전히 full-text, vector, hybrid search를 모두 선택지로 두고, chunking·enrichment·embedding·retrieval을 각각 독립적으로 평가하도록 안내한다. 4

읽는 순서와 보조 자료

34편을 위에서부터 순서대로 읽기보다 **의존관계 순서**로 읽는 것이 낫다.

가장 먼저 읽을 **핵심 골격 트랙**은 다음이다.

```
BM25 and Beyond
      ↓
DPR
      ↓
```

```
BEIR
      ↓
SPLADE v2
      ↓
HNSW
      ↓
Product Quantization
      ↓
BERT Passage Re-ranking
      ↓
LLM Re-ranking
      ↓
RRF
```

이 트랙을 마치면 다음 질문에 스스로 답할 수 있어야 한다.

> BM25와 dense retrieval은 무엇이 다른가?
> Sparse와 dense는 왜 함께 쓰는가?
> embedding model과 vector DB는 왜 별개의 문제인가?
> HNSW에서 recall을 높이면 무엇을 지불하는가?
> PQ는 왜 필요한가?
> retriever와 reranker는 왜 합치지 않고 분리하는가?
> BM25 score와 cosine similarity를 어떻게 합치는가?

DPR은 dense retrieval의 전환점을, HNSW/PQ는 vector-search system의 내부를, BERT reranking은 two-stage ranking architecture를, BEIR는 "한 데이터셋에서 잘되는 모델이 곧 범용 retriever는 아니다"라는 generalization 문제를 보여준다. 58

그다음 **RAG 데이터·context 트랙**은 이렇게 읽는 것을 권한다.

```
LayoutLMv3
    ↓
Nougat
    ↓
Dense X Retrieval
    ↓
Late Chunking
    ↓
Lost in the Middle
    ↓
RECOMP
```

이 순서의 목적은 "PDF → text → chunk → embedding → context"가 단순 ETL이 아니라는 것을 체감하는 것이다. 문서 layout이 깨지면 chunk가 깨지고, chunk가 깨지면 embedding이 잘못되고, 좋은 chunk를 가져와도 context 배치와 길이가 나쁘면 generator가 제대로 활용하지 못한다. Layout-aware document understanding, retrieval granularity, contextual chunk embedding, long-context utilization 연구가 이 연쇄관계를 각각 다른 위치에서 다룬다. 59

세 번째 **Query → Retrieval 최적화 트랙**은 다음이 좋다.

```
HyDE
  ↓
Query2doc
  ↓
RRF
  ↓
RAG-Fusion
  ↓
Cross-Encoder Reranking
  ↓
LLM Reranking
```

여기서는 검색 성능을 `embedding model` 하나의 문제로 보지 않는 습관을 들이는 것이 목표다. 같은 corpus와 같은 embedding model이라도 query rewrite, query expansion, candidate fusion, reranking을 바꾸면 final context가 크게 바뀔 수 있다. [60]

네 번째 **Evaluation 트랙**은 RAGAS → ARES → RAGChecker 순서가 좋다. 이때 반드시 일반 IR metric도 병행해서 공부해야 한다.

```
Retrieval
├─ Recall@K
├─ Precision@K
├─ MRR
└─ nDCG

Generation
├─ Answer Correctness
├─ Relevance
├─ Faithfulness / Groundedness
├─ Completeness
└─ Citation / Attribution

System
├─ End-to-end accuracy
├─ Latency
├─ Cost
├─ Throughput
└─ Security
```

Microsoft도 현재 retrieval step과 language-model end-to-end step을 따로 평가하고, groundedness·completeness·utilization·relevancy 같은 지표를 구분하도록 권고한다. RAGAS, ARES, RAGChecker의 연구 방향도 이런 **module-level diagnosis**와 맞닿아 있다. [61]

마지막 **Advanced RAG 트랙**은 이 순서가 가장 자연스럽다.

```
Original RAG (Lewis et al.)
       ↓
Multi-Hop Dense Retrieval
       ↓
IRCoT
       ↓
Self-RAG
     ↙    ↘
   CRAG   Adaptive-RAG
     \     /
       Agentic RAG
         ↓
GraphRAG / HippoRAG
```

Original RAG가 parametric/non-parametric memory 결합을 보여줬다면, multi-hop retrieval은 여러 검색 단계가 필요한 문제를, IRCoT는 retrieval과 reasoning의 interleaving을, Self-RAG/Adaptive-RAG/CRAG는 retrieval policy 자체의 동적 제어를 다룬다. GraphRAG와 HippoRAG는 flat vector space만으로는 포착하기 어려운 global structure와 associative multi-hop relation을 graph에 명시적으로 담는 방향을 보여준다. 62

전체 지도를 옆에 펼쳐놓을 **보조 reference**로는 Microsoft의 현행 RAG architecture guide가 특히 좋다. 이 가이드는 preparation → chunking → chunk enrichment → embedding → information retrieval → end-to-end evaluation을 한 흐름으로 정리하고, vector/full-text/hybrid search 및 agentic RAG까지 연결한다. 2026년 6월 30일 기준으로 업데이트된 문서이므로 현재 시스템 설계 체크리스트로 쓰기 좋다. 4

Microsoft — Design and develop a RAG solution
https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/rag/rag-solution-design-and-evaluation-guide

논문 지도를 분류학적으로 보고 싶다면 Gao et al.의 **Retrieval-Augmented Generation for Large Language Models: A Survey**가 좋다. 이 survey는 RAG를 Naive, Advanced, Modular RAG라는 흐름으로 보고 retrieval·generation·augmentation·evaluation을 연결한다. 63

Gao et al. — Retrieval-Augmented Generation for Large Language Models: A Survey
https://arxiv.org/abs/2312.10997

그리고 구현하면서 논문 기법을 하나씩 재현하는 보조 자료로는 `RAG_Techniques` 저장소가 유용하다. Semantic chunking, proposition chunking, HyDE, fusion retrieval, reranking, contextual compression, Self-RAG, CRAG, GraphRAG, RAPTOR, agentic 계열을 notebook 형태로 묶고 있어 위 논문 목록과 실습을 연결하기 좋다. 64

NirDiamant — RAG_Techniques
https://github.com/NirDiamant/RAG_Techniques

결국 이 논문 지도를 **한 장짜리 RAG 지식 트리**로 압축하면 다음과 같다.

```
RAG
 |
```

```
├── Data
│   ├── Parsing ............ LayoutLMv3, Nougat
│   ├── Cleaning ........... Deduplication, RefinedWeb
│   ├── Chunking ........... Dense X, Late Chunking
│   └── Metadata ........... Metadata Enrichment, Structure-Aware RAG
│
├── Representation
│   ├── Dense .............. DPR
│   └── Sparse ............. BM25 → SPLADE
│
├── Index
│   ├── Graph ANN .......... HNSW
│   └── Compression ........ PQ / IVF-PQ
│
├── Query
│   ├── Expansion .......... Query2doc
│   ├── Hypothetical ....... HyDE
│   └── Decomposition ...... Multi-hop / IRCoT
│
├── Retrieval
│   ├── Sparse ............. BM25 / SPLADE
│   ├── Dense .............. DPR
│   ├── Hybrid ............. Sparse + Dense
│   └── Benchmark .......... BEIR
│
├── Ranking
│   ├── Fusion ............. RRF / RAG-Fusion
│   ├── Cross-Encoder ...... BERT reranker
│   └── LLM ................ LLM reranking
│
├── Context
│   ├── Ordering ........... Lost in the Middle
│   └── Compression ........ RECOMP
│
├── Generation
│   ├── RAG ................ Lewis et al.
│   └── Attribution ........ ALCE
│
├── Evaluation
│   ├── RAG Metrics ........ RAGAS
│   ├── Judge Evaluation ... ARES
│   └── Diagnostics ........ RAGChecker
│
├── Advanced
│   ├── Self-reflection .... Self-RAG
│   ├── Correction ......... CRAG
│   ├── Routing ............ Adaptive-RAG
│   └── Agentic ............ Retrieval as Tool
```

```
 │
├── Structured Retrieval
│   ├── GraphRAG ........... Corpus-level structure
│   └── HippoRAG ........... Associative graph retrieval
 │
├── Multi-hop
│   ├── MDR ................ Iterative dense retrieval
│   └── IRCoT .............. Retrieval ↔ Reasoning
 │
└── Production
    ├── Latency / Cost ..... RAGCache
    └── Security ........... PoisonedRAG
```

가장 중요한 12편만 먼저 압축해서 읽는다면 `BM25 and Beyond → DPR → BEIR → SPLADE v2 → HNSW → Product Quantization → Passage Re-ranking with BERT → Dense X Retrieval → Lost in the Middle → RAGAS → Original RAG → Self-RAG` 순서가 가장 좋다. 이 12편이면 **전통 IR → sparse/dense retrieval → vector index → two-stage ranking → chunk/context → evaluation → RAG architecture → adaptive RAG**라는 전체 연구사의 뼈대가 잡힌다. 나머지 논문들은 이 뼈대의 특정 failure mode를 해결하는 가지로 읽으면 된다. [65]

---

[1] [4] [61] https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/rag/rag-solution-design-and-evaluation-guide
https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/rag/rag-solution-design-and-evaluation-guide

[2] [5] [6] [7] [53] [58] [65] https://aclanthology.org/2020.emnlp-main.550/
https://aclanthology.org/2020.emnlp-main.550/

[3] [31] [56] [60] https://aclanthology.org/2023.acl-long.99/
https://aclanthology.org/2023.acl-long.99/

[8] [16] https://arxiv.org/html/2603.04532v1
https://arxiv.org/html/2603.04532v1

[9] [10] SPLADE v2: Sparse Lexical and Expansion Model for Information Retrieval
https://arxiv.org/abs/2109.10086?utm_source=chatgpt.com

[11] [12] [54] Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs
https://arxiv.org/abs/1603.09320?utm_source=chatgpt.com

[13] Product quantization for nearest neighbor search.
https://pubmed.ncbi.nlm.nih.gov/21088323/?utm_source=chatgpt.com

[14] Billion-scale similarity search with GPUs
https://arxiv.org/abs/1702.08734?utm_source=chatgpt.com

[15] https://arxiv.org/html/2608.14886
https://arxiv.org/html/2608.14886

[17] https://scholar.google.co.kr/citations?hl=ko&user=0RAmmIAAAAAJ
https://scholar.google.co.kr/citations?hl=ko&user=0RAmmIAAAAAJ

18 https://aclanthology.org/people/dawei-yin/
https://aclanthology.org/people/dawei-yin/

19 https://aclanthology.org/2024.emnlp-main.845/
https://aclanthology.org/2024.emnlp-main.845/

20 https://arxiv.org/abs/2409.04701
https://arxiv.org/abs/2409.04701

21 Ragas: Automated Evaluation of Retrieval Augmented ...
https://arxiv.org/abs/2309.15217?utm_source=chatgpt.com

22 https://arxiv.org/abs/2311.09476
https://arxiv.org/abs/2311.09476

23 https://arxiv.org/abs/2408.08067
https://arxiv.org/abs/2408.08067

24 59 https://arxiv.org/abs/2204.08387
https://arxiv.org/abs/2204.08387

25 https://arxiv.org/abs/2308.13418
https://arxiv.org/abs/2308.13418

26 https://arxiv.org/abs/2408.09869
https://arxiv.org/abs/2408.09869

27 https://arxiv.org/abs/2107.06499
https://arxiv.org/abs/2107.06499

28 https://arxiv.org/abs/2306.01116
https://arxiv.org/abs/2306.01116

29 https://arxiv.org/abs/2512.05411
https://arxiv.org/abs/2512.05411

30 https://arxiv.org/abs/2605.24366
https://arxiv.org/abs/2605.24366

32 https://arxiv.org/abs/2303.07678
https://arxiv.org/abs/2303.07678

33 https://research.google/pubs/reciprocal-rank-fusion-outperforms-condorcet-and-individual-rank-learning-methods/
https://research.google/pubs/reciprocal-rank-fusion-outperforms-condorcet-and-individual-rank-learning-methods/

34 https://arxiv.org/abs/2402.03367
https://arxiv.org/abs/2402.03367

35 55 https://aclanthology.org/2024.tacl-1.9/
https://aclanthology.org/2024.tacl-1.9/

36 https://arxiv.org/abs/2310.04408
https://arxiv.org/abs/2310.04408

37 https://arxiv.org/abs/2005.11401
https://arxiv.org/abs/2005.11401

38 https://ai.meta.com/research/publications/retrieval-augmented-generation-for-knowledge-intensive-nlp-tasks/
https://ai.meta.com/research/publications/retrieval-augmented-generation-for-knowledge-intensive-nlp-tasks/

39 57 https://aclanthology.org/2023.emnlp-main.398/
https://aclanthology.org/2023.emnlp-main.398/

40 41 https://arxiv.org/abs/2404.12457
https://arxiv.org/abs/2404.12457

42 https://arxiv.org/html/2402.07867v1
https://arxiv.org/html/2402.07867v1

43 https://arxiv.org/html/2504.21668v2
https://arxiv.org/html/2504.21668v2

44 52 https://arxiv.org/abs/2310.11511
https://arxiv.org/abs/2310.11511

45 https://arxiv.org/abs/2403.14403
https://arxiv.org/abs/2403.14403

46 https://arxiv.org/abs/2401.15884
https://arxiv.org/abs/2401.15884

47 https://www.microsoft.com/en-us/research/publication/from-local-to-global-a-graph-rag-approach-to-query-focused-summarization/
https://www.microsoft.com/en-us/research/publication/from-local-to-global-a-graph-rag-approach-to-query-focused-summarization/

48 https://arxiv.org/abs/2405.14831
https://arxiv.org/abs/2405.14831

49 https://github.com/osu-nlp-group/hipporag
https://github.com/osu-nlp-group/hipporag

50 https://arxiv.org/abs/2009.12756
https://arxiv.org/abs/2009.12756

51 https://arxiv.org/abs/2212.10509
https://arxiv.org/abs/2212.10509

62 https://arxiv.org/html/2005.11401v4
https://arxiv.org/html/2005.11401v4

63 Retrieval-Augmented Generation for Large Language Models: A Survey
https://arxiv.org/abs/2312.10997?utm_source=chatgpt.com

64 NirDiamant/RAG_Techniques: This repository showcases ...
https://github.com/NirDiamant/rag_techniques?utm_source=chatgpt.com